

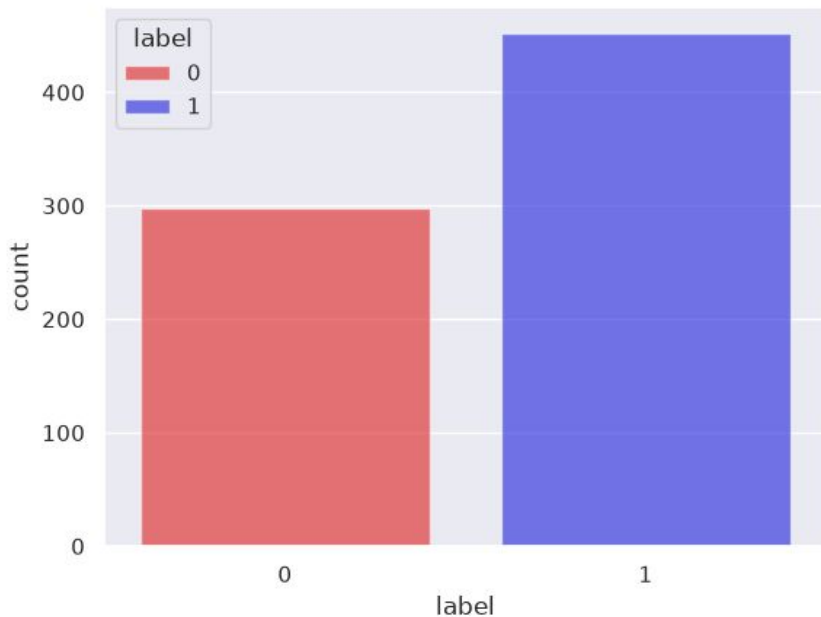
Predicting which songs Andreas likes

FRTN65 Lab 1, Andreas Olsson, October 2026

In short

- Data: 736 songs after dropping 14 duplicates, 13 features (10 numerical, 3 categorical), 60% liked
- Modelling: 12 methods tried in 51 variants, all through one evaluation protocol
- Result: four tree ensembles (xgboost, random forest, bagging, boosting) are indistinguishable on this data, around 0.83 accuracy
- The rule I set up before seeing any numbers picks bagging, the simplest of the equally good ones
- Handed in: random forest on the 10 numerical features, 0.833 ± 0.018 cross-validated accuracy. Why not bagging is on slide 13

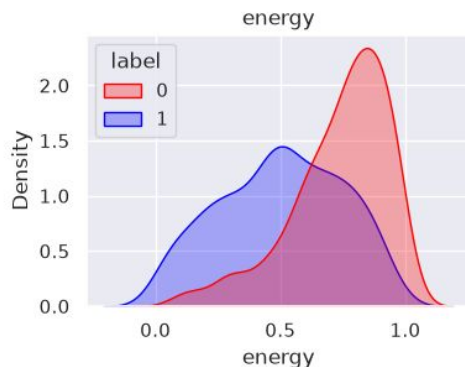
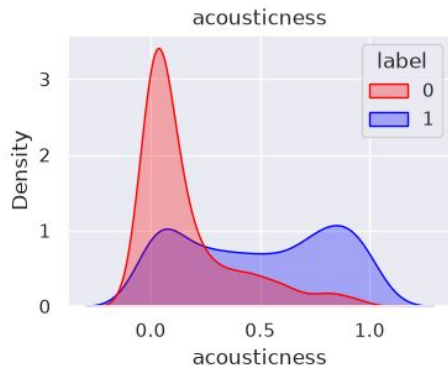
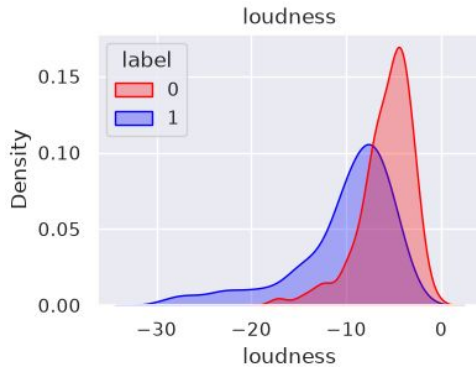
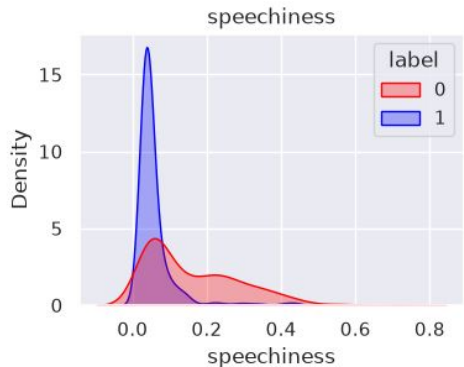
The data



Class balance in the raw file, 0 = dislike, 1 = like.

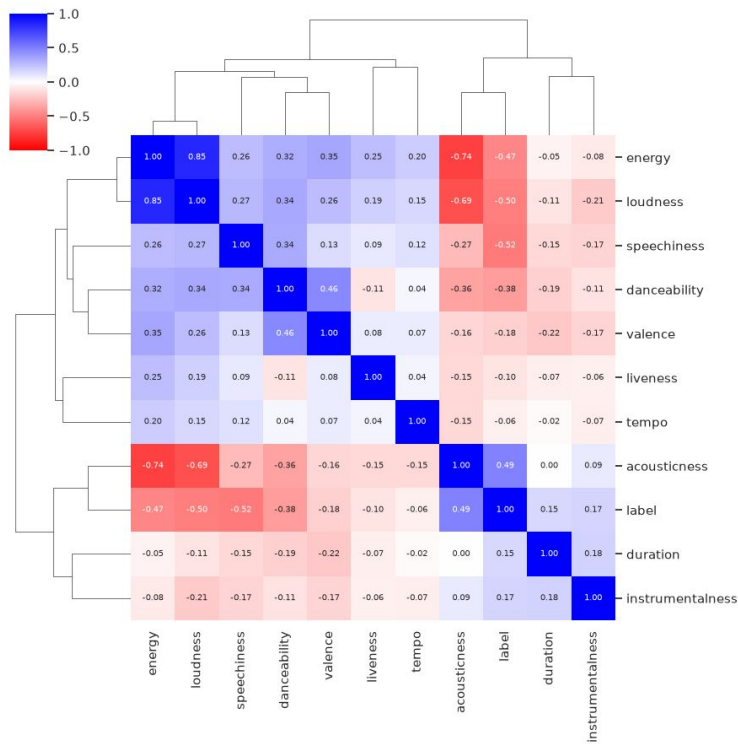
- 750 rows, 14 exact duplicates dropped, 736 songs left. No missing values
- 13 Spotify audio features per song
- 10 numerical: acousticness, danceability, duration, energy, instrumentalness, liveness, loudness, speechiness, tempo, valence
- 3 categorical: key, mode, time_signature
- 443 liked, 293 disliked. Always guessing "like" gives 0.60 accuracy, so that is the floor every number is measured from

The data, the four features that matter



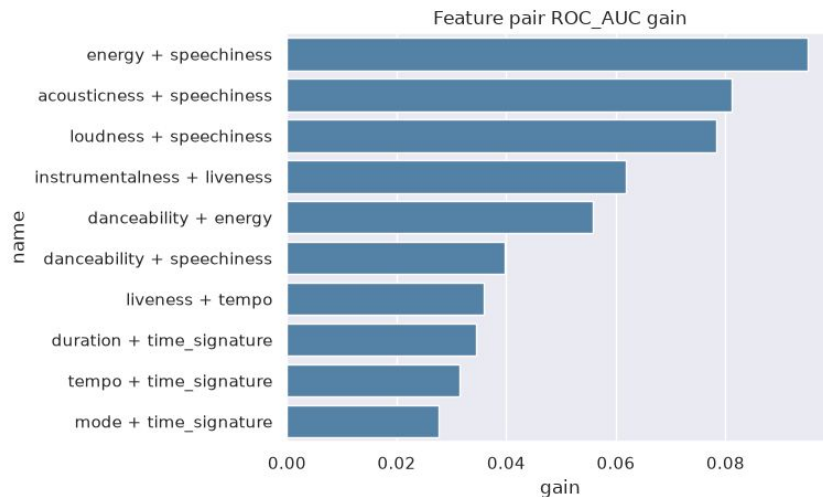
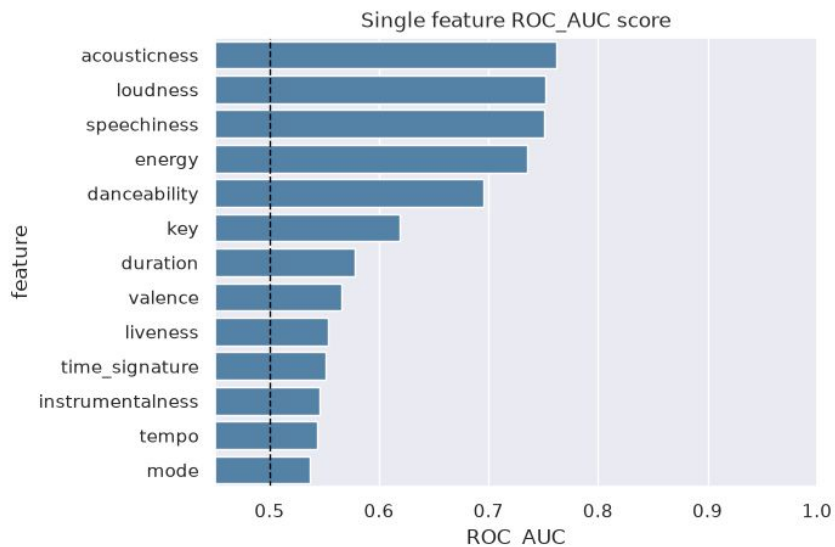
- Liked songs are quiet, acoustic and have almost no speech. Disliked songs are loud, energetic and speechy
- The densities are per class, so the 60/40 does not show up as separation here
- speechiness: the liked songs are one spike near zero, the disliked ones spread out into a tail. duration, instrumentalness and liveness have the same spike plus tail shape, which is why scaling becomes a question later

Feature correlations



- Spearman (rank) correlation, bc of the tails. Clustered so the blocks show
- energy, loudness and acousticness are one block (energy to loudness 0.85, energy to acousticness -0.74): three columns, one direction
- speechiness has the strongest single relation to the label (-0.52) and sits outside that block
- tempo, liveness, duration, instrumentalness: near zero against everything
- Correlation only sees pairwise monotone relations, not interactions, so the next slide asks a decision tree instead

Feature screen with decision trees



A depth-limited decision tree, 5-fold cross validation, ROC AUC, on each feature alone (left) and on each pair (right). Gain = pair minus the better of its two singles. Every single feature is above 0.5, and every big pair gain has speechiness in it.

Preprocessing

- All 13 features go in by default: the 10 numerical ones standardised (z-score), the 3 categorical ones one-hot. 28 columns reach the model
- Scaling only matters for the methods that measure distances or penalise coefficients (knn, svm, logreg, lda, qda). Trees do not care, so they get the raw columns
- Variants on top of that: robust and quantile scaling, log1p on the spiky four, rare levels grouped, categoricals dropped, only the four strong features, pairwise products of the numerical ones
- All of it lives inside the sklearn Pipeline, so scaler and encoder are fit on the training part of each fold only, never on the validation part (important)
- The test file is read once, at the very end, by the model that was handed in

The methods

method	what it is	tuned by the grid search
dummy	always answers "like", the floor	nothing
logreg	one linear boundary, L2 penalty on the coefficients	C
lda	linear too, gaussian classes with one shared covariance	nothing
qda	a covariance per class, so a quadratic boundary	reg_param
knn	vote among the k closest songs in the scaled space	k, weights
tree	one tree of threshold splits, readable, high variance	max_depth, min leaf
svm_linear	like logreg but hinge loss, maximises the margin	C
svm_rbf	kernel trick, a nonlinear boundary	C, gamma
bagging	300 trees on bootstrap samples, averaged	min leaf
rf	bagging plus a random feature subset at every split	max_features, min leaf
boosting	shallow trees in sequence, each fixing the last ones' errors	learning rate, depth, iterations
adaboost	boosting with stumps and exponential loss	learning rate, n trees
xgboost	gradient boosting again, (outside the course list)	learning rate, depth, min child weight

The evaluation protocol

- Metric: accuracy, bc that is what the leaderboard measures. Balanced accuracy and ROC AUC are reported beside it but never decided on
- Splits: 5-fold stratified cross validation, repeated 5 times with one seed, so 25 outer splits. Every method sees exactly the same 25, so every comparison is paired
- Tuning: grid search with 5 inner folds inside the outer training part only. The validation fold never sees the search (nested cross validation)
- Uncertainty: the 25 splits share most of their songs, so $\text{std} / \sqrt{25}$ is too small. I use the Nadeau-Bengio corrected standard error, $\text{std} \cdot \sqrt{1/25 + n_{\text{val}} / n_{\text{train}}}$, on the scores and on the paired differences

Deciding between equally good methods

- Candidates: every variant within one corrected standard error of the best (the error of the paired gap, not of the means)
- Among the candidates: the one with the fewest tuned hyperparameters. Ties by an order fixed in advance: a line < a density model < a distance model < one tree < a kernel < an ensemble of trees
- Why fewest tuned and not fewest parameters: among methods the data cannot tell apart, I prefer the one whose score had the fewest chances to look good by search. Parameter count measures capacity, and for ensembles it is a bad guide (more trees do not overfit more)
- The band (one error) and the rule were written down before any number came out. slide 12 shows what two errors, or best mean alone, would have picked instead

Results, 5 repeats

variant	accuracy $\pm$ corrected error	balanced accuracy	ROC AUC	num hyps tuned
xgboost	0.834 $\pm$ 0.016	0.824	0.905	3
rf_numeric (hand-picked)	0.833 $\pm$ 0.018	0.825	0.912	2
rf	0.833 $\pm$ 0.018	0.825	0.911	2
bagging (rule-picked)	0.828 $\pm$ 0.017	0.819	0.905	1
boosting	0.826 $\pm$ 0.015	0.814	0.902	3
svm_rbf_top4	0.824 $\pm$ 0.016	0.808	0.878	2
knn_top4	0.824 $\pm$ 0.015	0.805	0.880	2
svm_rbf_numeric	0.821 $\pm$ 0.016	0.804	0.881	2
logreg_elastic	0.817 $\pm$ 0.018	0.805	0.888	1
svm_rbf	0.814 $\pm$ 0.016	0.801	0.887	2
svm_linear	0.813 $\pm$ 0.017	0.800	0.888	1
adaboost	0.809 $\pm$ 0.017	0.796	0.890	2
logreg	0.807 $\pm$ 0.019	0.794	0.887	1
knn	0.802 $\pm$ 0.012	0.776	0.883	2
tree	0.800 $\pm$ 0.017	0.786	0.845	2
lda	0.795 $\pm$ 0.016	0.782	0.879	0
qda	0.783 $\pm$ 0.017	0.788	0.879	1

Mean over the 25 outer splits $\pm$ Nadeau-Bengio corrected standard error. Bold: what the rule picks. Blue: what was handed in. dummy (always "like"): 0.602.

Paired against the best, and the rule applied

variant	gap to xgboost	corrected error	within 1 error	within 2 errors
rf_numeric	-0.001	0.007	yes	yes
rf	-0.001	0.007	yes	yes
bagging	-0.006	0.008	yes	yes
boosting	-0.008	0.009	yes	yes
svm_rbf_top4	-0.010	0.013	yes	yes
knn_top4	-0.010	0.016	yes	yes
svm_rbf_numeric	-0.013	0.013	no	yes
logreg_elastic	-0.017	0.011	no	yes
svm_rbf	-0.020	0.011	no	yes
svm_linear	-0.021	0.010	no	no
logreg	-0.027	0.012	no	no
lda	-0.039	0.011	no	no

The rule (one error, fewest tuned hyperparameters) picks bagging: seven candidates, bagging tunes one hyperparameter. At two errors logreg_elastic joins and the rule would pick it instead. Best mean alone would pick xgboost.

Why I handed in rf_numeric and not bagging

- Under the rule: bagging. That string is submission_best_choice.txt
- The leaderboard counts accuracy only, so for the competition I took the highest expected score among the methods the data cannot tell apart, and gambled
- rf_numeric is the forest without key, mode and time_signature. Same mean as the full forest (gap $+0.000 \pm 0.007$ on the screen, 0.833 for both at 5 repeats) and the best balanced accuracy and ROC AUC of all the course methods (0.825, 0.912)
- Dropping the three categorical columns costs nothing the data can detect, and every diagnostic rates them at zero (slide 14)
- Not xgboost: outside the course list, and not better than the forest by anything measurable
- The two strings differ on 10 of the 200 test songs

What every method leans on

permutation importance: drop in held-out accuracy when the feature is shuffled

feature	variant					
	logreg	lda	knn	svm_rbf	rf	boosting
speechiness	0.096	0.113	0.091	0.098	0.090	0.102
acousticness	0.024	0.024	0.047	0.026	0.025	0.043
loudness	0.025	0.001	0.015	0.027	0.027	0.033
danceability	0.008	0.011	0.037	0.028	0.010	0.024
energy	0.004	0.018	0.018	0.015	0.010	0.012
duration	0.008	0.000	0.009	0.004	0.008	0.011
instrumentalness	0.006	0.003	0.008	0.005	0.000	0.008
liveness	-0.001	-0.000	0.007	0.007	0.003	0.005
valence	-0.001	-0.001	-0.002	0.005	0.005	0.006
tempo	-0.006	-0.002	0.004	0.002	0.003	0.016
key	0.000	0.002	0.001	0.002	-0.001	0.004
time_signature	0.002	0.002	-0.000	0.002	-0.001	-0.000
mode	-0.004	-0.001	0.004	-0.001	-0.002	0.000

- Permutation importance: the drop in held-out accuracy when one feature is shuffled, per method
- speechiness is the one feature every method leans on (0.09 to 0.11), then acousticness and loudness
- key, mode and time_signature are zero for every method. This is the evidence for dropping them in rf_numeric
- knn and boosting also use danceability a bit. Nobody uses tempo, valence or liveness

Conclusions

- Four tree ensembles tie at 0.83 and nothing beats them. By the rule the simplest of them, bagging, wins. For the competition, the forest on the numerical features
- The data has one strong axis (loud and energetic against quiet and acoustic) plus speechiness. A heavily regularised line already gets 0.81, pairs and trees buy the last two points
- Scaling only matters where distances or penalties are, and the wrong scaler costs four points. The three categorical columns carry nothing
- Pairing the splits and correcting the standard error is what made "equally good" a statement instead of a feeling

Generative AI was used improve the visuals in certain plots, and with some statistical details, like the corrected standard error. I also used it to exemplify the difference of the normal kind of “exploratory university notebooks” (like the one given) and more professional grade ML projects, to give me better real-life context. It was also used to suggest an outline for this presentation, given my own notebooks.